Evidencia 2: Estructuras lineales y aplicación de logs

Documento de evidencia

Diego Contreras Alvarez, A00845716

Programación de Estructuras de Datos y Algoritmos Fundamentales (TC1031)

Grupo 607

Profesor: David Alonso Cantú Delgado

Fecha: 18 de octubre de 2026

Nota sobre los archivos de entrada

Las instrucciones hablan de log608-1.txt y log608-2.txt, pero los archivos que se entregaron se llaman log607-1.txt y log607-2.txt, así que en este documento se usan con ese nombre. Además la descripción de las instrucciones viene al revés. Revisando los archivos se ve que log607-1.txt es el que está en orden aleatorio (empieza con Sep 29 2024, luego Feb 13 2026 y luego May 04 2025) y log607-2.txt es el casi ordenado (empieza con Sep 08 2024 a las 00:22, 04:37 y 05:50). Todo lo que sigue usa esa descripción real. El archivo de salida sí conserva el nombre output608.out que piden las instrucciones.

1. Evidencia de ejecución

1.1 Compilación

La aplicación y el programa de pruebas se compilan por separado, porque cada uno tiene su propio main. Los dos compilan sin advertencias con -Wall -Wextra.

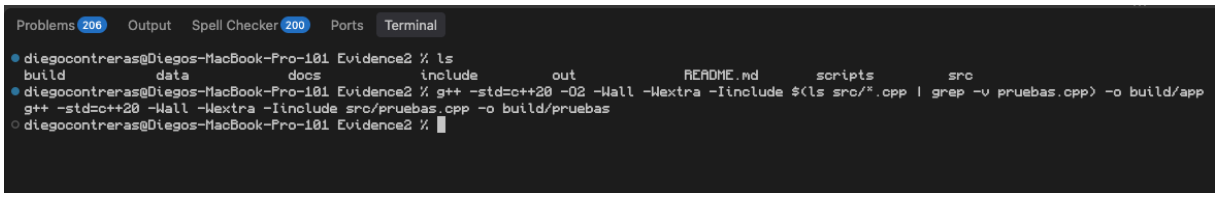


Figura 1. Compilación de la aplicación y del programa de pruebas sin advertencias.

1.2 Programa de pruebas de la Parte A

El programa de pruebas (src/pruebas.cpp) revisa las cuatro estructuras en el mismo orden: vacía, un solo elemento, varias inserciones y eliminaciones en los extremos, e impresión. Cada revisión escribe [OK] o [FALLA] y al final se muestra el total. La corrida completa hace 140 revisiones con 0 fallas, y el texto completo de la salida está en docs/salida_pruebas.txt.

En la LinkedList se prueba que sacar de una lista vacía no cierre el programa y que tail se reinicie bien después de vaciarla. En la Pila se ve el orden LIFO, porque después de push(1, 2, 3) el primer pop saca el 3. En la Fila se ve el orden FIFO, porque después de enqueue(1, 2, 3) el primer dequeue saca el 1. En la lista doblemente ligada, después de cada cambio se recorre hacia adelante con next y hacia atrás con prev, y se revisa que los dos recorridos salgan al revés uno del otro. Si algún prev quedara mal, solo se notaría en el recorrido hacia atrás.

```
diegocontreras@Diegos-MacBook-Pro-101 Evidence2 % g++ -std=c++20 -O2 -Wall
g++ -std=c++20 -Wall -Wextra -Iinclude src/pruebas.cpp -o build/pruebas
diegocontreras@Diegos-MacBook-Pro-101 Evidence2 % ./build/pruebas
Pruebas de estructuras - Evidencia 2
Diego Contreras Alvarez, A00845716

==================== LinkedList ====================

Prueba 1. Lista vacia
  [OK] isEmpty() es verdadero
  [OK] size() = 0
  removeFront: la lista esta vacia, no hay nada que quitar
  [OK] removeFront sobre vacia regresa false y no cambia el valor
  [OK] getFront sobre vacia regresa false

Prueba 2. Un solo elemento
  [OK] insertBack(10) en vacia deja size() = 1
  [OK] el primero es 10
  [OK] removeFront saca el 10
  [OK] al quitar el unico nodo la lista queda vacia
  [OK] insertFront(20) en vacia funciona despues de vaciarla

Prueba 3. Insertar y quitar varias veces en los extremos
  [OK] insertFront(0,1) e insertBack(2,3) dejan 4 elementos
  Lista: 0-1-2-3
  [OK] removeFront saca el 0
  [OK] insertBack(4) y removeFront saca el 1
  [OK] se sacan 2, 3 y 4 y queda vacia
  removeFront: la lista esta vacia, no hay nada que quitar
  [OK] insertBack(7) despues de vaciarla (tail se reinicio bien)

Prueba 4. Imprimir
  Lista vacia:
  Lista 5, 6, 7: 5-6-7

==================== Stack ====================

Prueba 5. Pila vacia
  [OK] isEmpty() es verdadero
  [OK] size() = 0
  pop: la pila esta vacia
  [OK] pop sobre vacia regresa false y no cambia el valor
  peek: la pila esta vacia
  [OK] peek sobre vacia regresa false y no cambia el valor

Prueba 6. Un solo elemento
  [OK] push(10) deja size() = 1
  [OK] peek ve el 10
  [OK] peek no saca el elemento
  [OK] pop saca el 10
  [OK] la pila queda vacia

Prueba 7. Insertar y quitar varias veces en el extremo (arriba)
  [OK] push(1,2,3) deja el 3 arriba
  [OK] pop saca el 3 (el ultimo que entro)
  [OK] push(4) y pop saca el 4
  [OK] pop saca el 2
  [OK] pop saca el 1 y queda vacia
  pop: la pila esta vacia

Prueba 8. Imprimir (de arriba hacia abajo)
  Pila vacia:
  Despues de push 5, 6, 7: 7-6-5
```

Figura 2. LinkedList (pruebas 1 a 4) y Stack con orden LIFO (pruebas 5 a 8).



Figura 3. Queue con orden FIFO (pruebas 9 a 12) e inicio de DoublyLinkedList.

```
diegocontreras@Diegos-MacBook-Pro-101 Evidence2 % ./build/pruebas
  [OK] insertBack(10) en vacia: head y tail son el mismo nodo
  Adelante: 10
  Atras:    10
  [OK] la lista queda 10
  [OK] hacia atras sale al reves que hacia adelante
  [OK] cada prev apunta al nodo de atras
  [OK] el ultimo nodo recorrido es el tail
  [OK] size() = 1
  remove del unico nodo:
  [OK] head y tail vuelven a nullptr
  Adelante: (vacia)
  Atras:    (vacia)
  [OK] la lista queda (vacia)
  [OK] hacia atras sale al reves que hacia adelante
  [OK] cada prev apunta al nodo de atras
  [OK] el ultimo nodo recorrido es el tail
  [OK] size() = 0
  [OK] insertFront(20) en vacia: head y tail son el mismo nodo
  Adelante: 20
  Atras:    20
  [OK] la lista queda 20
  [OK] hacia atras sale al reves que hacia adelante
  [OK] cada prev apunta al nodo de atras
  [OK] el ultimo nodo recorrido es el tail
  [OK] size() = 1

Prueba 15. insertAfter sobre el tail
  insertAfter(tail, 30):
  [OK] el 30 es el nuevo tail
  Adelante: 20 30
  Atras:    30 20
  [OK] la lista queda 20 30
  [OK] hacia atras sale al reves que hacia adelante
  [OK] cada prev apunta al nodo de atras
  [OK] el ultimo nodo recorrido es el tail
  [OK] size() = 2
  insertAfter(tail, 40):
  Adelante: 20 30 40
  Atras:    40 30 20
  [OK] la lista queda 20 30 40
  [OK] hacia atras sale al reves que hacia adelante
  [OK] cada prev apunta al nodo de atras
  [OK] el ultimo nodo recorrido es el tail
  [OK] size() = 3

Prueba 16. insertAfter en medio y en el head
  insertAfter(head, 25):
  Adelante: 20 25 30 40
  Atras:    40 30 25 20
  [OK] la lista queda 20 25 30 40
  [OK] hacia atras sale al reves que hacia adelante
  [OK] cada prev apunta al nodo de atras
  [OK] el ultimo nodo recorrido es el tail
  [OK] size() = 4
  insertAfter(nodo del 30, 35):
  Adelante: 20 25 30 35 40
  Atras:    40 35 30 25 20
  [OK] la lista queda 20 25 30 35 40
  [OK] hacia atras sale al reves que hacia adelante
  [OK] cada prev apunta al nodo de atras
  [OK] el ultimo nodo recorrido es el tail
  [OK] size() = 5
```

Figura 4. DoublyLinkedList: un solo elemento e insertAfter sobre el tail, en medio y en el head.



Figura 5. DoublyLinkedList: inserciones en los extremos y remove del head, del tail y de un nodo en medio.



Figura 6. DoublyLinkedList: quitar todo hasta el único nodo, impresión en los dos sentidos y resumen con 0 fallas.

1.3 Opciones del menú

Las corridas de esta sección se hicieron primero con log607-2.txt (casi ordenado) y al final con log607-1.txt (aleatorio), para que output608.out quedara generado con log607-1.txt.

Carga y opción 1 con log607-2.txt. Al iniciar, el programa pregunta qué archivo cargar, encola cada línea en la Fila, la procesa y la inserta al final de la lista doble. Reporta 6818 registros cargados y 0 líneas omitidas. Antes de ordenar pide la predicción del crecimiento de las comparaciones.

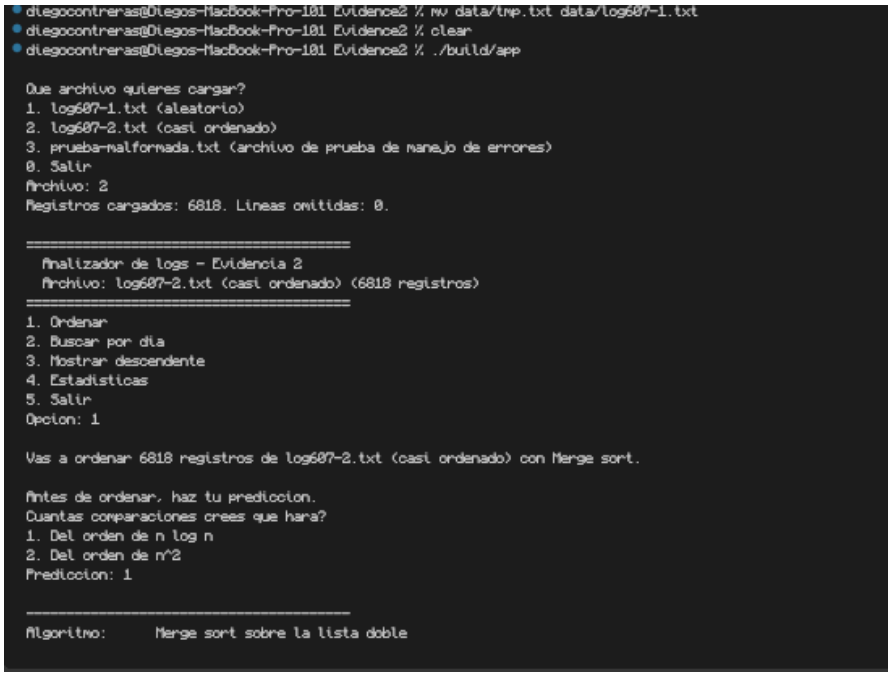


Figura 7. Carga de log607-2.txt y predicción antes de ordenar.

Opciones 1, 2 y 3. El resultado del ordenamiento muestra el algoritmo, el archivo, n, el tiempo, las comparaciones, la profundidad máxima y si coincidió la predicción. Al pedir otra vez la opción 1 el programa avisa que la lista ya está ordenada. La opción 2 busca Dec 16 2025 y encuentra 13 registros, y la opción 3 muestra esos mismos 13 registros del más reciente al más antiguo usando la Pila.



Figura 8. Resultado del ordenamiento de log607-2.txt, lista ya ordenada, búsqueda por día con formatos inválidos y vista descendente.

Opción 4. Las estadísticas se calculan con un solo recorrido de la lista. El día con más incidentes es Jan 27 2025 con 21 registros, el evento más frecuente es Malware Detection Alerts con 342, y para la IP hay un empate de 13 direcciones con 3 apariciones cada una, que se muestran todas.



Figura 9. Vista descendente, día sin registros, estadísticas con empate y salida con la opción 5.

Opción 1 con log607-1.txt. Esta es la última corrida, la que deja generado el archivo de salida.

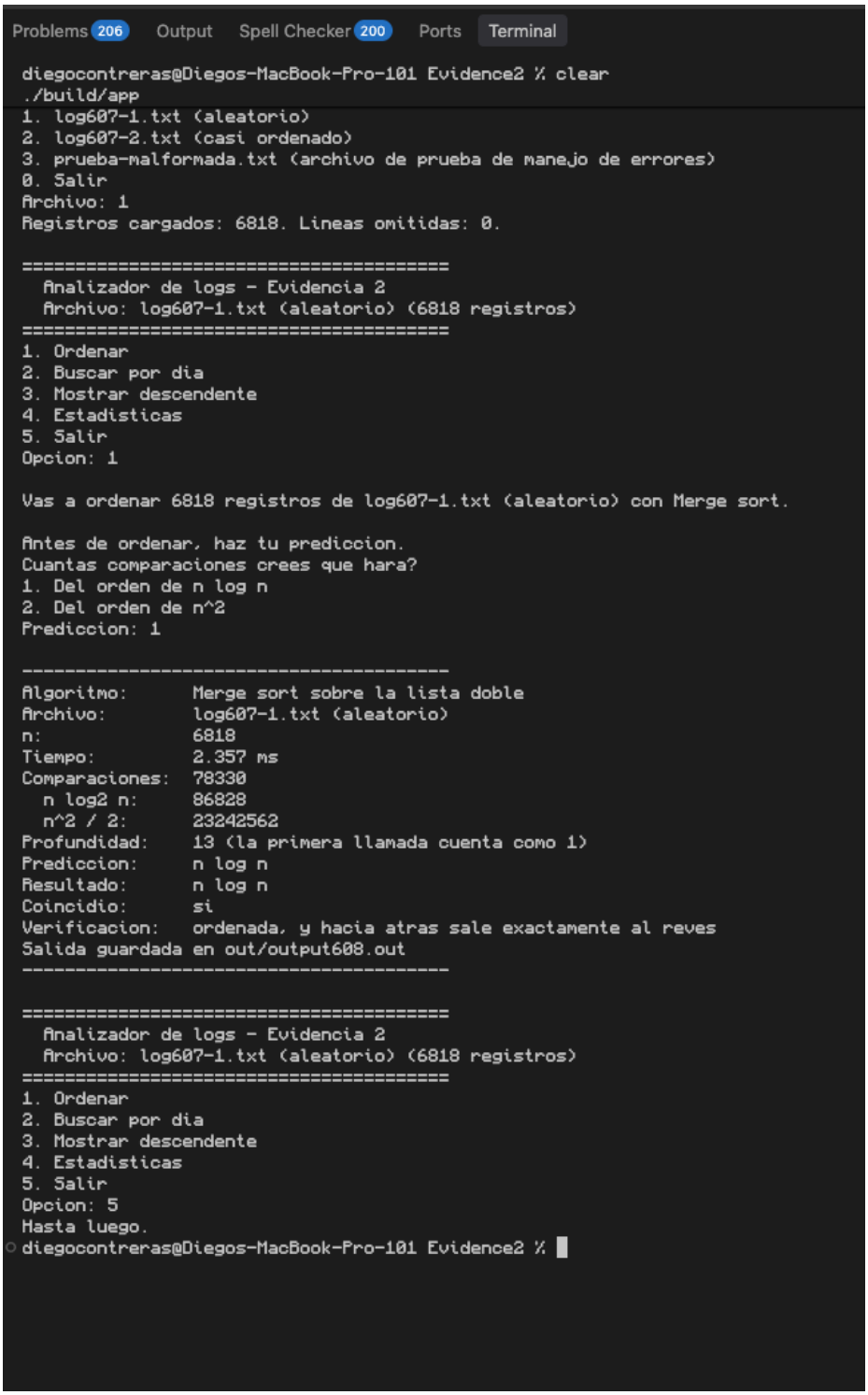


Figura 10. Carga y ordenamiento de log607-1.txt.

Parte del archivo output608.out. Se muestran las primeras y las últimas cinco líneas. El archivo tiene las 6818 líneas, empieza en el registro más antiguo (Sep 08 2024 00:22:43) y termina en el más reciente (Sep 07 2026 23:50:40), con el mismo formato que el archivo de entrada.

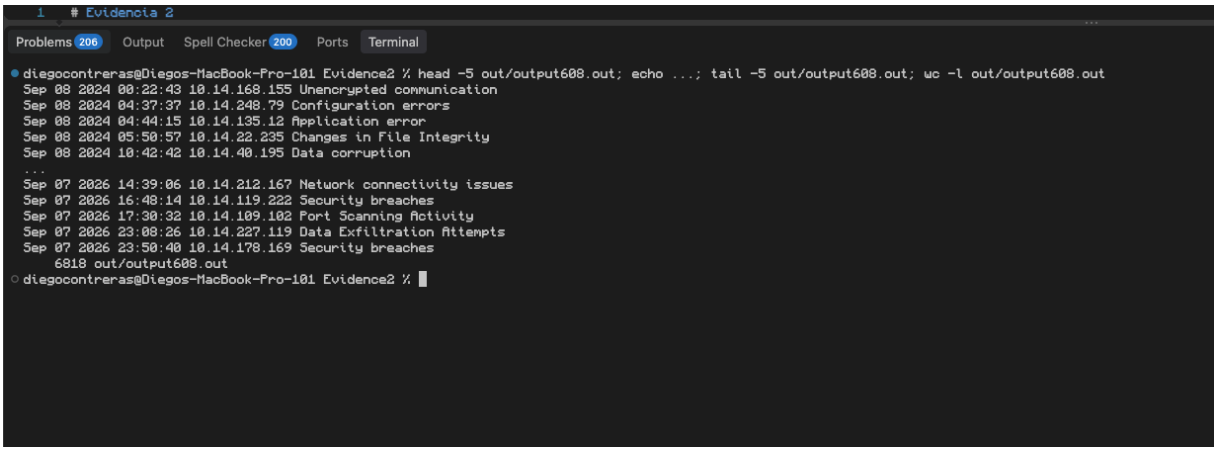


Figura 11. Primeras y últimas líneas de out/output608.out y su número de líneas.

1.4 Casos de borde

Caso	Entrada	Resultado obtenido	Figura
Archivo inexistente	log607-1.txt renombrado a tmp.txt, archivo 1	Muestra la ruta que no pudo abrir y vuelve a preguntar qué archivo cargar sin cerrarse	Figura 13
Línea con formato inválido	prueba-malformada.txt	Se omiten 4 de 6 líneas con su número y motivo: línea vacía, mes inválido (dos) y fecha imposible (Feb 30). Se cargan 2 registros	Figura 12
Día sin registros	Sep 01 2024	No hay registros para ese dia	Figura 9
Formato de día inválido	Feb 30 2025, Xyz 01 2025, Dec 16	Los tres se rechazan con el formato esperado y vuelve a pedir el día	Figura 8
Buscar antes de ordenar	Opción 2 sin ordenar	Primero ordena la lista con la opcion 1	Figura 12
Ordenar una lista ya ordenada	Opción 1 dos veces	La lista ya esta ordenada, no hay nada que hacer	Figura 8
Empates en las estadísticas	Opción 4	Muestra las 13 IPs empatadas con 3 apariciones	Figura 9
Opción de menú inexistente	9 y abc	Entrada invalida. Escribe un numero entre 1 y 5	Figura 12

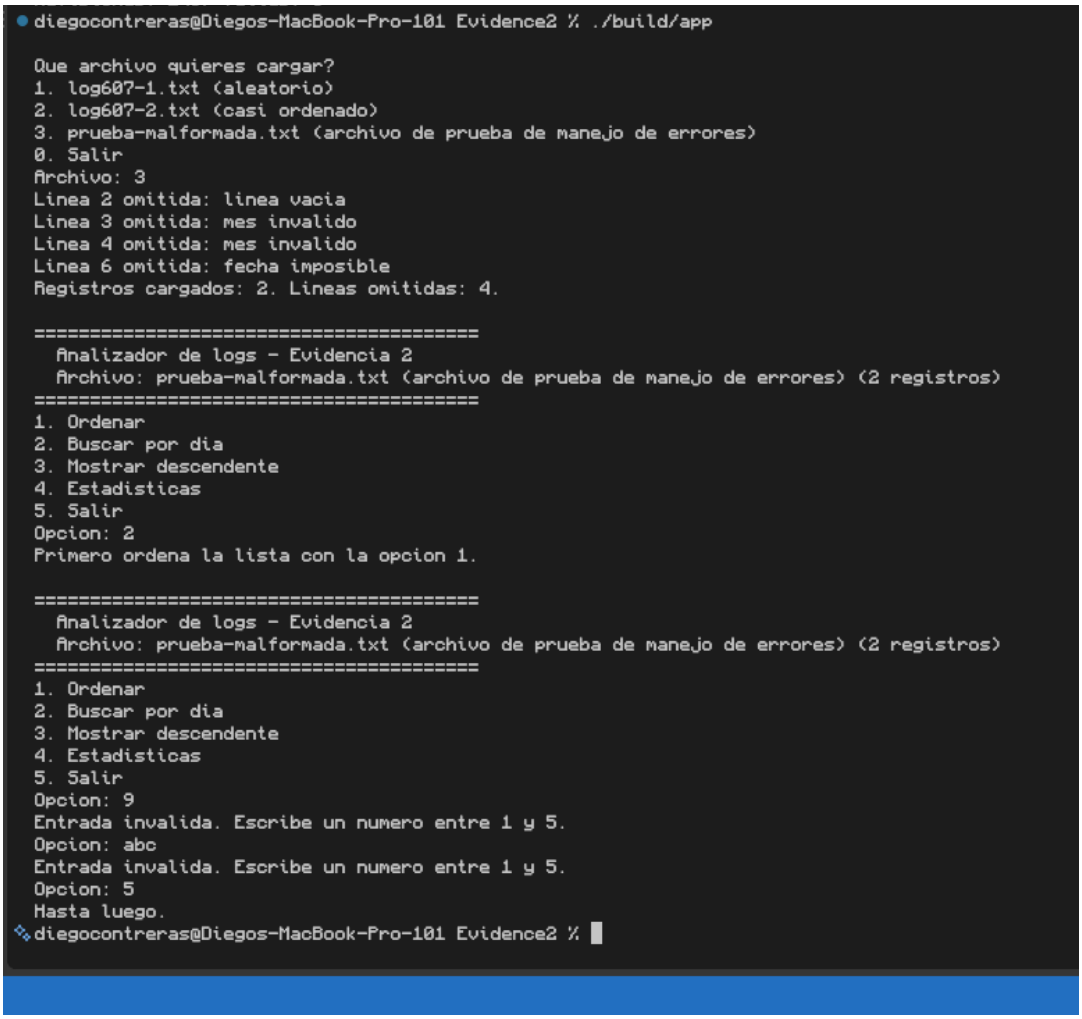


Figura 12. Líneas inválidas en prueba-malformada.txt, buscar antes de ordenar y opciones inexistentes.

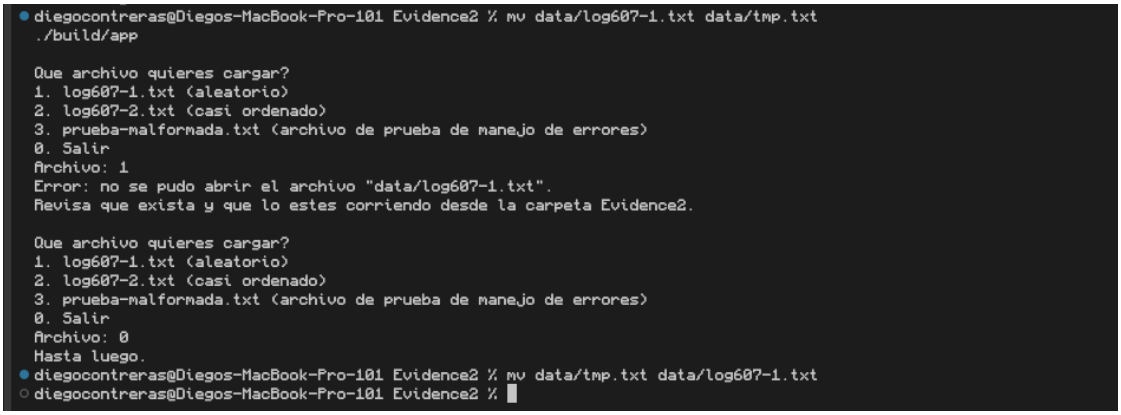


Figura 13. Archivo inexistente.

2. Resultados del ordenamiento

El algoritmo elegido fue Merge Sort, implementado directamente sobre la lista doblemente ligada. No se copian los registros a un arreglo ni se crean nodos nuevos, solo se cambian los apuntadores next y prev. Se eligió Merge Sort porque hace del orden de n log n comparaciones sin importar cómo vengan los datos y porque es estable. Quick Sort en una lista ligada no puede tomar la mediana de tres de forma

barata, porque llegar al nodo de en medio cuesta un recorrido, y con el pivote en un extremo se degrada a n^2 justo con el archivo casi ordenado.

Archivo	Predicción	Tiempo	Comparaciones	$n \log_2 n$	Profundidad máxima	Coincidió	Conservó el orden de llegada
log607-2.txt (casi ordenado)	n log n	1.733 ms	50,624	86,828	13	Sí	Sí
log607-1.txt (aleatorio)	n log n	2.357 ms	78,330	86,828	13	Sí	Sí

En los dos archivos n = 6818 y $n^2/2$ = 23,242,562, así que las comparaciones quedaron muy por debajo de n^2 y cerca de $n \log_2 n$. La profundidad cuenta la primera llamada como 1 y no cuenta las llamadas que reciben una cadena de un solo nodo, porque esas regresan de inmediato sin dividir nada.

Para comprobar que se conservó el orden de llegada de los registros con la misma fecha y hora, la salida se comparó contra la de sort -s, que es un ordenamiento estable de la terminal, usando como llave la fecha y la hora. El resultado fue de 0 diferencias. Esto importa porque los archivos traen 18 pares de registros con la misma fecha y hora, y un algoritmo inestable podría invertir alguno de esos pares. La comparación se hizo con los dos archivos, ordenando cada uno con la aplicación justo antes de compararlo, y en los dos dio 0 diferencias.



Figura 14. log607-1.txt: output608.out contra un ordenamiento estable, 0 diferencias.



Figura 15. log607-2.txt: output608.out contra un ordenamiento estable, 0 diferencias.

Interpretación

En log607-1.txt, que viene en orden aleatorio, Merge Sort hizo 78,330 comparaciones, un poco menos que $n \log_2 n$. Esto cuadra con la teoría, porque cada mezcla de dos mitades de tamaño k hace a lo mucho 2k menos 1 comparaciones y en datos aleatorios casi siempre se acerca a ese máximo.

En log607-2.txt, que viene casi ordenado, hizo 50,624 comparaciones, alrededor de 35 % menos. La razón está en cómo termina la mezcla. Cuando una de las dos mitades se acaba, lo que sobra de la otra se pega completo sin compararlo. Si los datos ya vienen casi en orden, en muchas mezclas toda la mitad izquierda es menor que la derecha, así que la izquierda se acaba después de k comparaciones y la derecha se engancha entera. El tiempo también bajó, de 2.357 ms a 1.733 ms. Aun así la diferencia es pequeña comparada con algoritmos como insertion sort, que en la Evidencia 1 cambiaba más de cien veces entre un archivo y otro. Merge Sort sigue dividiendo y mezclando lo mismo sin importar el orden de entrada.

La profundidad fue de 13 en los dos archivos. Eso pasa porque Merge Sort siempre parte la lista a la mitad sin ver los datos, así que la profundidad solo depende de n. Como 2^{12} = 4,096 es menor que 6,818 y 2^{13} = 8,192 es mayor, se necesitan 13 niveles de división para llegar a cadenas de un nodo.

3. Prompts utilizados

[Por escribir: copiar los 2 o 3 prompts principales, describir qué parte del código generaron o cómo ayudaron y qué se modificó, y describir al menos un caso concreto en el que la IA se equivocó, cómo se detectó y cómo se corrigió.]

4. Reflexión personal

[Por escribir, de 1 a 2 páginas. Debe cubrir estos puntos:]

- Qué aprendiste del uso de IA, qué parte hiciste tú, qué parte sugirió o completó la IA y qué habrías hecho diferente sin su ayuda.
- Por qué cada estructura ocupa su lugar en la aplicación, y si era necesaria la Pila para la opción 3 aunque la lista doble ya se pueda recorrer hacia atrás con prev, con ventajas y desventajas de cada opción.
- Por qué no se puede aplicar búsqueda binaria directamente sobre una lista ligada y qué costaría hacerlo posible.
- Interpretación de las mediciones: comportamiento en cada archivo y relación entre la profundidad de recursión y la pila de llamadas.